The complete, unified Git lifecycle. Think of this as your master blueprint, taking you from a blank screen to a fully synchronized, daily development workflow.

Before you can build anything with Git, it demands to know who you are.

git config --global user.name "Sachin Joshi"

git config --global user.email "sachinjoshi@gmail.com"

You must run these commands first for two major reasons:

- **The Passport Control Rule:** Git will literally refuse to save your work (commit) until you give it a name and email. If you skip this, Git will throw an error the moment you try to save code, blocking your progress.
- **The Credit Rule (The Green Squares):** GitHub uses your configured email address to connect your local Git Bash work to your online GitHub profile. If the email doesn't match, GitHub won't show your profile picture next to your code, and you won't get credit for your work on your contribution graph.

A Quick English Translation:

> *"Hey Git, change a global setting for my entire computer. Find the user name setting, and set it to '**Sachin Joshi.**"*

Phase 1: The Initial Setup (Choose Your Path)

You only need to do **one** of these paths once per project to map Git Bash to GitHub.

Path A: If the repository already exists on GitHub

Use this if you are downloading a project to start working on it. Git automatically maps everything for you.

1. Copy the repository URL from GitHub.
2. Open Git Bash and navigate to your folder: cd path/to/folder
3. Run the clone command:

```
Bash
git clone https://github.com/your-username/your-repo-name.git
```

4. Move into the newly created project folder:

```
Bash
cd your-repo-name
```

Path B: If the files only exist on your local computer

Use this if you started writing code locally and now want to back it up to a brand-new, empty GitHub repository.

1. Open Git Bash inside your local project folder and initialize Git:

```
Bash
git init
```

2. Stage and commit your existing files:

```
Bash
git add .
git commit -m "Initial commit"
```

3. Set your main branch name and map it to your new GitHub URL:

```
Bash
git branch -M main
git remote add origin https://github.com/your-username/your-repo-name.git
```

4. Push your local code up for the first time (linking the branches):

```
Bash
git push -u origin main
```

Phase 2: The Daily Workflow (The Standard Loop)

Once Phase 1 is done, your mapping is locked in. From this point forward, you will follow this exact **4-step loop** every single day you write code.

```
[ 1. PULL ] -> Get the latest code from GitHub

[ 2. WORK ] -> Write code / make changes locally

[ 3. STAGE & COMMIT ] -> Package your changes

[ 4. PUSH ] -> Upload back to GitHub
```

Step 1: PULL (Before you touch a single line of code)

Always start your session by grabbing the latest changes from GitHub. This prevents you from writing code on top of an outdated version.

```
Bash
git pull origin main
```

Step 2: WORK

Open your code editor, write your features, fix your bugs, and save your files as you normally would.

Step 3: STAGE & COMMIT (Package your work)

Once you reach a good stopping point, open Git Bash and prepare your files for upload.

- Check what changed: git status
- Stage all changes: git add .
- Lock them in with a message: git commit -m "Add a brief description of what you did"

Step 4: PUSH (Upload it)

Send your fresh commits up to GitHub for safekeeping (or for your team to see).

```
Bash
git push origin main
```

How to Verify It Worked

If you want to double-check that your local folder is correctly mapped to the right GitHub address, run this command in Git Bash:

```
Bash
git remote -v
```

You should see two lines showing your GitHub URL—one for (fetch) and one for (push). If you see them, you are officially mapped and ready to code!

The "Am I Synchronized?" Cheat Sheet

Keep these quick diagnostic commands in your back pocket to verify your flow:

- **git remote -v** -> Checks if your local folder is successfully mapped to GitHub.
- **git status** -> Tells you if your files are staged, unstaged, or if your local branch is ahead/behind GitHub.
- **git log --oneline** -> Shows a quick history of your recent commits so you can see what has been saved.

💡 The Golden Rule of Git

Always git pull before you start working, and always git pull before you git push. If someone else has uploaded code to GitHub and you try to push your changes before pulling theirs, GitHub will reject your push with an error. Pulling first ensures your computer knows about the latest changes, allowing Git to cleanly combine everyone's work.